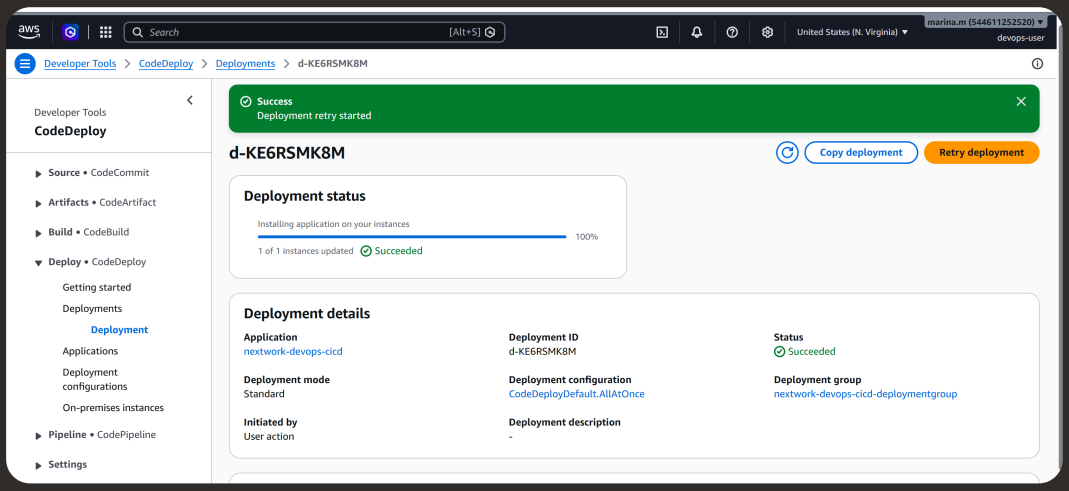
DEPLOY A WEB APP WITH CODEDEPLOY

By Maryna Maksimuk | October 2026



Introducing Today's Project!

Today, I will use AWS CodeDeploy to deploy the packaged application from Amazon S3 to web servers, making the web application accessible to users.

Key tools and concepts

I learned how to use AWS CodeDeploy, CodeBuild, S3, EC2, IAM, and CloudFormation to automate application deployment. I also learned about CI/CD pipelines, deployment artifacts, appspec.yml, lifecycle scripts, IAM roles and permissions, and troubleshooting deployment failures.

Project reflection

It took me approximately 3 hours to complete the project, including configuring the AWS services, troubleshooting deployment issues, and successfully deploying the application.

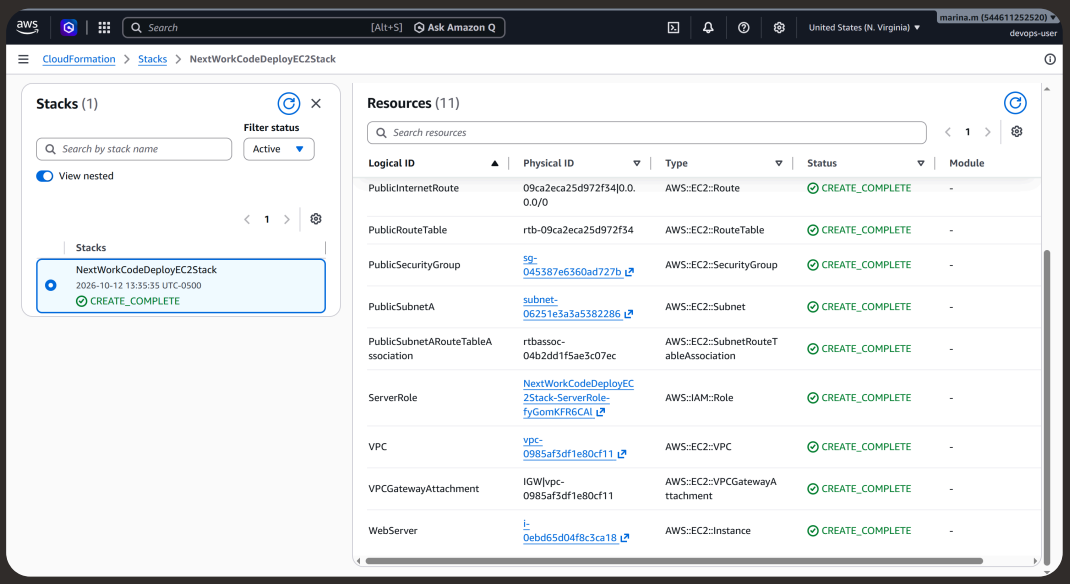
I plan to start the next project tomorrow to continue building my AWS and DevOps skills through hands-on practice.

Deployment Environment

I launched a VPC to create an isolated network for my AWS resources and an EC2 instance to run applications within that network. This helped me understand how CloudFormation uses Infrastructure as Code to create and manage cloud infrastructure consistently and automatically.

I used AWS CloudFormation to deploy the EC2 instance and VPC using an Infrastructure as Code template.

CloudFormation created a VPC, public subnet, Internet Gateway, route table, and internet route to configure network connectivity for the EC2 instance. It also created a security group, IAM role, instance profile, and the EC2 web server, allowing the server to communicate securely and access the AWS resources it needs.



Deployment Scripts

Scripts are files containing commands that automatically tell a computer what to do, such as installing software, stopping a server, or starting a web application, instead of performing those steps manually. In your AWS CodeDeploy project, the scripts will help automate the process of preparing the EC2 instance and running your application during deployment.

The 'install_dependencies.sh' will install Tomcat to run your Java web application and Apache HTTP Server to receive incoming web requests. It configures Apache to forward requests on port 80 to your application running on Tomcat at port 8080.

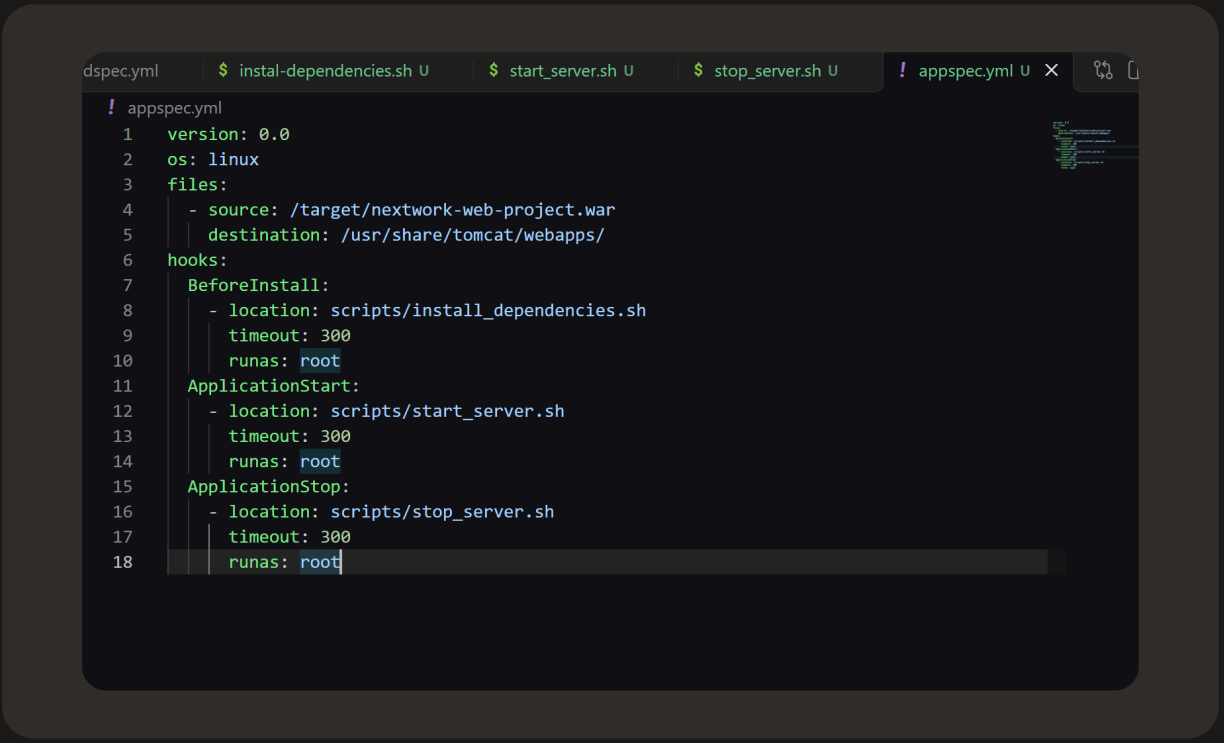
The start_server.sh will start the Tomcat and Apache (httpd) services immediately and enable them to start automatically whenever the EC2 instance reboots. Tomcat runs your Java web application, while Apache receives incoming web requests and forwards them to Tomcat.

The stop_server.sh checks whether Apache (httpd) and Tomcat are running and, if they are, stops both services to prepare the EC2 server for a new deployment. If either service is not running, the script skips stopping that service and continues to the next check.

appspec.yml

appspec.yml is a configuration file used by AWS CodeDeploy that tells it which application files to copy, where to place them, and which scripts to run during deployment, such as stopping the server, installing the application, and starting the services again.

I updated buildspec.yml to include the deployment scripts and appspec.yml in the build artifact so AWS CodeDeploy can use them to stop services, deploy the application, and restart the server

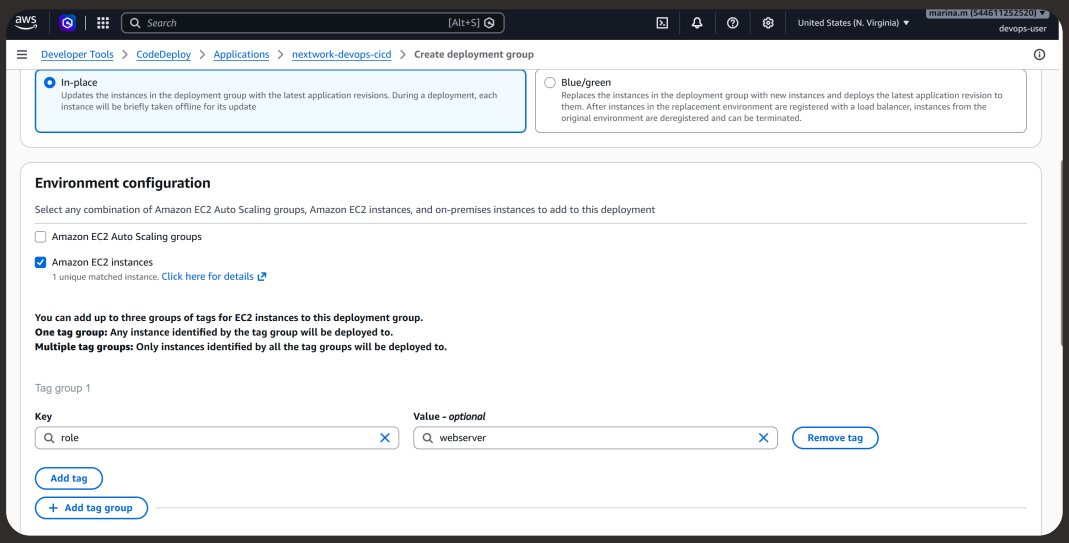


Setting Up CodeDeploy

A CodeDeploy application represents the application you want to deploy, while a deployment group defines the deployment settings, such as which EC2 instances receive the application and how the deployment is performed. Application as the software we are deploying and the deployment group as the set of rules that controls where and how it gets deployed.

CodeDeploy needs an IAM service role so it has permission to access the AWS resources required for deployment, such as EC2 instances and deployment files stored in Amazon S3. The role allows CodeDeploy to perform these actions securely without using your personal AWS credentials.

We use tags to identify which EC2 instances CodeDeploy should deploy the application to. For example, if we tag an instance with role=webserver, we can configure the deployment group to target only instances with that tag, making deployments easier to manage and control.



Deployment configurations

AWS CodeDeploy offers different deployment configurations that control how and how quickly an application is deployed.

AllAtOnce: Deploys to all instances at the same time. Fast, but riskier if something goes wrong.

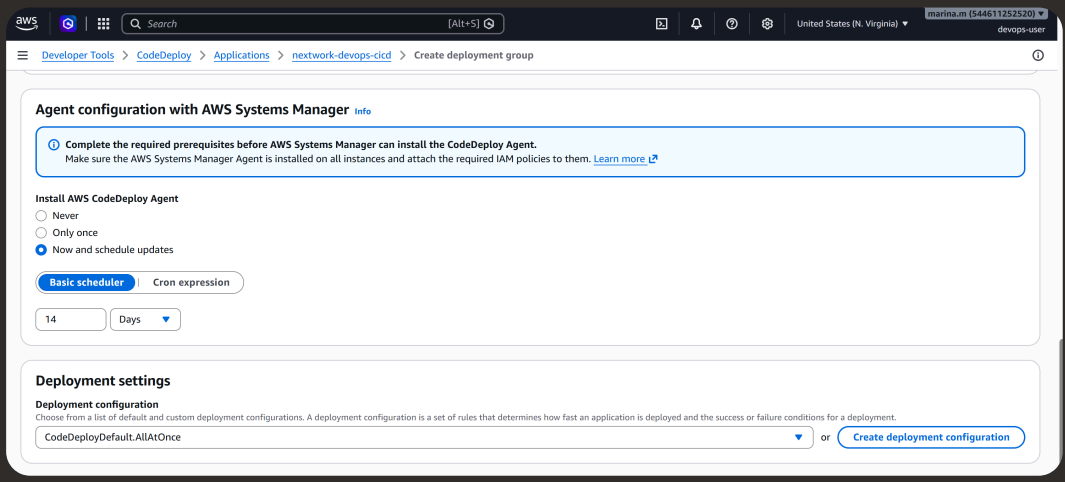
HalfAtATime: Deploys to up to half the instances at a time, keeping some instances available.

OneAtATime: Deploys to one instance at a time, reducing risk but taking longer.

Custom configurations: Let you specify the minimum number or percentage of healthy instances that must remain available during deployment.

For your EC2 project, these options help balance deployment speed, application availability, and risk.

The CodeDeploy Agent is software installed on an EC2 instance that communicates with AWS CodeDeploy and carries out deployments. It downloads the application files, copies them to the correct locations, and runs deployment scripts such as stopping and restarting services.



Success!

A CodeDeploy deployment is the process of delivering and installing an application on target servers, such as an EC2 instance. CodeDeploy uses the application files, appspec.yml, and deployment scripts to copy the files, configure the application, and start the required services.

A revision location is where AWS CodeDeploy finds the application files it needs to deploy, usually a ZIP file stored in an Amazon S3 bucket or a GitHub repository. In your project, the revision location points to the S3 artifact containing your Java web application, appspec.yml, and deployment scripts.

I verified that the CodeDeploy deployment status was Succeeded in the AWS console. I also checked that the application was deployed to the EC2 instance and that the web application was accessible in the browser.

